

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 6

Operating Systems

Topic: Inside the xv6 File System

Checked by:

Reițman Patricia, university assistant

Chișinău — 2026

CONTENTS

INTRODUCTION 3
1 XV6 FILES AND DIRECTORIES 4
2 MYSTAT AND HARD LINKS 5
3 FILE SYSTEM LIMITS 7
4 RECURSIVE FIND 8
CONCLUSION 9
REFERENCES 10

INTRODUCTION

This laboratory studies xv6 file-system behavior from both its shell and its C source. I implemented mystat to inspect inode metadata and find to traverse directory entries recursively, then tested hard links, long names and the on-disk layout. The results below follow my saved xv6 terminal record; an initial reused disk image caused a directory-creation failure, so the final find test was run after make clean.

Figure note. The terminal-style figures reproduce selected excerpts from the saved terminal text and are not original screenshots. Numerical results are preserved as recorded; the code blocks condense relevant logic from my source files.

1 XV6 FILES AND DIRECTORIES

This chapter follows Part 1 of the handout. I decoded the columns of xv6 ls and inspected a directory already present in the file-system image.

1.1 Directory entries and file types

```
ls
mkdir a
ls a
```

The evidence for this exercise is shown in Figure 1.1.



Figure 1.1 – The xv6 root and an existing directory

Comment and observation

kernel/stat.h maps type 1 to directory, type 2 to file and type 3 to device. The root inode is 1, so root's '.' and '..' both refer to inode 1: the parent of root is root itself. An empty a is 32 bytes because it contains two 16-byte directory entries, '.' and '..' (a two-byte inode number plus a 14-byte name). mkdir a failed on the first boot because a was already present in the reused file-system image.

2 MYSTAT AND HARD LINKS

This chapter follows Part 2 of the handout. I implemented mystat, tested link counts before and after unlinking, and inspected the inode definitions in the xv6 source.

2.1 The mystat implementation

```
char *typename(int t) {
  if(t == T_DIR) return "directory";
  if(t == T_FILE) return "file";
  if(t == T_DEVICE) return "device";
  return "?";
}
for(int i = 1; i < argc; i++) {
  if(stat(argv[i], &st) < 0) {
    fprintf(2, "mystat: cannot stat %s\n", argv[i]);
    continue;
  }
  printf("%s: %s, inode %d, links %d, size %d bytes\n",
         argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
}
```

The program calls stat for each pathname and prints the type, inode number, hard-link count and byte size. A failed lookup reports to descriptor 2 and continues with later arguments. I added $U/_mystat to Makefile's UPROGS list; the build log shows user/_mystat was produced. One initial invocation printed 'exec mystat failed'; a repeated invocation produced the successful output below. The record does not establish the cause of the first failure.

```
mystat README . console
ln README r2
mystat README r2
rm README ; mystat r2
```

The evidence for this exercise is shown in Figure 2.1.



Figure 2.1 – mystat reports inode and link-count changes

Comment and observation

After ln, README and r2 both identified inode 2 and both showed two links. After rm README, r2 still identified inode 2 and its link count fell to one; the inode and 2441-byte contents did not change. cat r2 | wc returned 48 lines, 336 words and 2441 bytes. A separate lookup of nosuchfile correctly reported an error.

2.2 Unlinking and the on-disk inode

```
grep -n 'nlink' kernel/sysfile.c
grep -n -B2 -A7 'struct dinode' kernel/fs.h
```

The evidence for this exercise is shown in Figure 2.2.



Figure 2.2 – Relevant source lines from the student's tree

Comment and observation

In sys_unlink, line 244 decrements the unlinked inode's nlink; line 239 separately adjusts a parent directory's count when a subdirectory is removed. The inode and data blocks can be reclaimed after no directory links and no live open references remain. The on-disk dinode stores type, major and minor device numbers, nlink, size and data-block addresses. This xv6 exercise has no Unix permission bits or symbolic-link type in that structure.

3 FILE SYSTEM LIMITS

This chapter follows Part 3 of the handout. I compared a long pathname with DIRSIZ, calculated the maximum file size and examined the mkfs disk layout.

3.1 Name and file-size limits

```
grep -n 'define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ' kernel/fs.h
mkdir averyveryverylongname ; ls
```

The evidence for this exercise is shown in Figure 3.1.

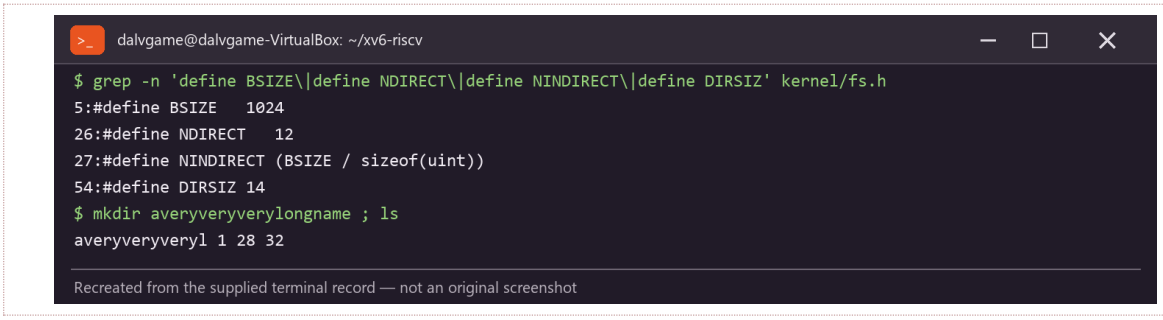


Figure 3.1 – Constants and truncated directory name

Comment and observation

ls showed averyveryveryl, the first 14 characters of the requested name; DIRSIZ is 14. With 12 direct blocks and 1024/4 = 256 indirect addresses, the largest file is (12 + 256) × 1024 = 274,432 bytes, or 268 KiB, in this single-indirect layout.

```
make clean ; make fs.img 2>&1 | grep nmeta
```

The evidence for this exercise is shown in Figure 3.2.



Figure 3.2 – mkfs disk layout

Comment and observation

The image totals 2000 blocks. Metadata uses 47 blocks: one boot block, one superblock, 31 log blocks, 13 inode blocks and one bitmap block. The remaining 1953 blocks are the data region. The long-name test and the later clean build were separate boots, so inode numbers can differ between them.

4 RECURSIVE FIND

This chapter follows Part 4 of the handout. I implemented recursive directory traversal and tested the optional inode-number output on a clean disk.

4.1 Recursive directory traversal

```
if(st.type == T_DIR) {
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)) {
    if(de.inum == 0) continue;
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0) continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    find(buf, name, showino);
  }
}
close(fd);
```

find opens a path, uses fstat to determine its type, compares the final pathname component with the target, and recursively reads nonempty directory entries. It skips '.' and '..' to avoid a cycle, checks buffer space, closes opened descriptors and accepts the optional -n flag for inode numbers. I registered $U/_find in Makefile; the build log shows user/_find was produced.

```
make clean ; make qemu
mkdir a
mkdir a/b
echo x > a/b/r2
echo y > a/r2
ln cat r2
find . r2
find a b
find . r2 -n
```

The evidence for this exercise is shown in Figure 4.1.



Figure 4.1 – Selected results from the clean-disk find test

Comment and observation

The clean-disk test found both nested r2 files and the root-level hard link. find a b returned a/b. With -n, ./r2 was inode 3, matching cat; the nested files had distinct inodes 30 and 31. An earlier echo failed, but the repeated command succeeded.

CONCLUSION

The implemented tools exposed xv6's compact file-system model. mystat showed that hard-link names share one inode while the link count changes. This connected the shell commands to the on-disk metadata studied in the source code.

Source constants explained the 14-character directory-name limit, the 268 KiB maximum file and the 2000-block disk layout. The clean-disk find tests showed recursive traversal, avoidance of '.' and '..', and identification of cat's hard link by inode number.

REFERENCES

OS 2026 Lab 6 handout, Inside the xv6 File System, provided for this course. Source files consulted in the student's xv6 tree: kernel/stat.h, kernel/fs.h and kernel/sysfile.c.